

● CHULETA MVVM + ROOM + LIVE DATA (PROYECTO PRODUCTOS)

MVVM + Room + LiveData - Proyecto Productos

1 Modelo (`Producto.kt`)

```
```kotlin
@Entity(tableName = "productos")
data class Producto(
 @PrimaryKey(autoGenerate = true)
 val id: Int = 0,
 val nombre: String,
 val categoria: String
)
```

- Define la tabla productos.
- id autogenerado, nombre y categoria campos básicos.

---

### 2 DAO (ProductoDao.kt)

```
@Dao
interface ProductoDao {
 @Insert
 fun insertar(producto: Producto)

 @Query("SELECT * FROM productos")
 fun getAll(): LiveData<List<Producto>>
}
```

- DAO que Room implementa automáticamente.
- LiveData permite observar cambios sin corrutinas.

---

### 3 Base de datos (AppDataBase.kt)

```
@Database(entities = [Producto::class], version = 1, exportSchema = false)
abstract class AppDataBase : RoomDatabase() {
 abstract fun productoDao(): ProductoDao
}
```

```

companion object {
 @Volatile
 private var INSTANCE: AppDataBase? = null

 fun getDatabase(context: Context): AppDataBase {
 return INSTANCE ?. synchronized(this) {
 val instance = Room.databaseBuilder(
 context.applicationContext,
 AppDataBase::class.java,
 "producto_db"
)
 .allowMainThreadQueries() // Solo para práctica
 .build()
 INSTANCE = instance
 instance
 }
 }
}

```

- Patrón Singleton.
- `allowMainThreadQueries()` solo para prácticas.

#### 4 Repository (Repository.kt)

```

class Repository(private val dao: ProductoDao) {
 fun insertar(producto: Producto) {
 dao.insertar(producto)
 }

 fun listaTodo(): LiveData<List<Producto>> = dao.getAll()
}

```

- Intermediario entre DAO y ViewModel.

#### 5 ViewModel (ProductoViewModel.kt)

```

class ProductoViewModel(application: Application) : AndroidViewModel(application) {
 private val repository: Repository
 val listaProductos: LiveData<List<Producto>>
}

```

```

init {
 val dao = AppDataBase.getDatabase(application).productoDao()
 repository = Repository(dao)
 listaProductos = repository.listaTodo()
}

fun insertar(producto: Producto) {
 repository.insertar(producto)
}
}

```

- Expone LiveData a la UI.
- insertar() delega en Repository.

---

## 6 Activity (MainActivity.kt)

```

class MainActivity : AppCompatActivity() {

 private lateinit var binding: ActivityMainBinding
 private val viewModel: ProductoViewModel by viewModels()

 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 binding = ActivityMainBinding.inflate(layoutInflater)
 setContentView(binding.root)

 // Observar LiveData
 viewModel.listaProductos.observe(this) { lista ->
 // Actualizar UI (RecyclerView, TextView, etc.)
 }

 // Insertar ejemplo
 val producto = Producto(nombre = "Teclado", categoria = "Periférico")
 viewModel.insertar(producto)
 }
}

```

- by viewModels() instancia automáticamente el ViewModel.
- LiveData notifica automáticamente a la UI cada vez que cambia la base de datos.

---

## 7 Flujo MVVM Simplificado

UI (Activity/Fragment)

|

v

ViewModel (ProductoViewModel)

|

v

Repository

|

v

Room (AppDataBase + ProductoDao)

|

v

LiveData → Activity observa cambios automáticamente

- Insertar: Activity → ViewModel → Repository → DAO → Room → LiveData → UI
- Consultar: Activity observa LiveData → se actualiza automáticamente